

우테코 BACKEND - LEVEL 1 → LEVEL 2 회고

컨트롤러의 오해, 그리고 서비스 계층

계층별 필요성과 목적, 그리고 주의점.
“근거 없는 감각이 아니라, 문제의 정체를 언어로 정의하기”

블랙잭 미션에서 시작된 작은 위화감이 계층 구조 전체를 다시 보게 만든 과정

모카

01 위화감의 시작

블랙잭 미션 컨트롤러가 너무 무겁다

03 가벼워졌지만 찝찝함

기대했던 웹 컨트롤러의 모습이 아니다

05 계층 개념 다시 세우기

MVC패턴, Repository/Service계층은 무엇을 위해 왜 나왔는가

07 해소된 네가지 문제

컨트롤러에 뭉쳐 있던 유스케이스를 Service로 뽑아내자 한번에 풀렸다.

09 다시 바라보는 문제 정의

무엇이 진짜 문제였는가?

11 왜 ‘계층’ 개념이 혼란스러운가?

비슷한 개념에 붙여진 여러 다른 이름들

02 무겁다는 감각만으로 시작한 리팩토링

전략/콜백 패턴으로 컨트롤러 가볍게 만들기

04 컨트롤러 오개념 바로잡기

콘솔 MVC의 컨트롤러는 원래 무겁다

06 서비스 계층의 근본 이유

주요 탄생 이유와 해소된 네 가지 문제

08 서비스가 두꺼워지는 함정

도메인은 왜 아무것에도 의존하지 않는게 좋은가?

10 서비스 입력 Command DTO의 발견

웹환경에 콘솔 환경을 더하며 느낀 필요성

01

위화감의 시작

레벨 1 블랙잭 미션 - “왜 컨트롤러가 이렇게 무겁지?”

블랙잭 미션 - 컨트롤러가 모든 걸 떠안았다

초기 BlackJackController는 게임의 처음부터 끝까지 모든 흐름을 직접 들고 있었다.

BlackJackController (초기)

```
public void run() {
    Players players = inputView.readPlayers();
    Dealer dealer = new Dealer();
    Deck deck = new Deck();

    BlackJackGame game =
        startGame(players, dealer, deck);
    playGame(game);           // 입력 루프
    showGameResult(game, ...); // 출력
}
```

```
// 입력 + 흐름제어 + 도메인조립 + 출력
// 이 한 클래스가 전부 담당
```

컨트롤러 하나가 떠안은 책임

- 사용자 입력 받기와 파싱
- 게임 객체 생성과 조립
- 플레이어 / 딜러 턴 진행 루프
- 턴마다 화면 출력 호출
- 최종 수익 판정 흐름 제어

느낀 점

“컨트롤러는 거쳐 가는 곳일 텐데, 왜 이렇게 할 일이 많지?”

“아 컨트롤러가 블랙잭 게임 비즈니스 규칙을 모르게 하고싶은데..”

02

무겁다는 감각만으로 시작한 리팩토링

전략 패턴과 콜백으로 일단 컨트롤러를 가볍게 만들어보기 - [java-blackjack / step2](#)

전략 패턴과 콜백으로 흐름을 도메인에 넘기다

“무겁다”는 감각만으로, 입력 판단과 출력 시점을 도메인이 부르도록 뒤집었다.

BlackJackGame (도메인이 흐름 보유)

```
public void proceedAllPlayersTurn(
    PlayerHitStrategy hitStrategy,
    Consumer<Player> onCardReceived) {
    for (Player player : players.getPlayers()) {
        while (player.shouldDraw()
            && hitStrategy.shouldHit(player)) {
            player.receiveCard(deck.draw());
            onCardReceived.accept(player); // 콜백
        }
    }
}
```

BlackJackController (step2)

```
private void playGame(BlackJackGame game) {
    game.proceedAllPlayersTurn(
        new ConsolePlayerHitStrategy(inputView),
        outputView::printCard);

    game.proceedDealerTurn(
        new DefaultDealerHitStrategy(),
        outputView::printDealerDraw);
}

// 루프가 사라지고 위임만 남음
```

전략 패턴 - “카드를 더 받을까?” 판단을 객체로 분리해 도메인에 주입. **콜백** - “언제 출력할까?” 시점을 도메인이 역으로 호출.

03

가벼워졌지만, 이상하다 째름하다

컨트롤러는 분명 가벼워졌지만, 기대했던 컨트롤러의 모습이 아니다.

비우긴 했는데, 오히려 이해하기 어려워졌다

성공한 것

- 컨트롤러 코드 줄 수는 확실히 줄었다
- 입력 판단·출력 시점이 객체로 분리됐다
- 감각적으로는 “가벼워졌다”

그런데 이상한 것

- 흐름이 도메인/전략/콜백에 흩어져 추적이 어렵다
- “게임 진행”을 읽으려면 여러 파일을 오가야 한다
- 기대한 웹 컨트롤러의 깔끔함과는 다른 모습

핵심 질문

“컨트롤러를 가볍게 만들었는데 왜 더 어지러울까?”
비우는 방향은 맞았을지 몰라도, 무엇을 어디로 왜 어떤걸 위해 옮길지를 몰랐다.

04

컨트롤러 오개념 바로잡기

내가 알던 '컨트롤러'는 사실 콘솔 MVC의 컨트롤러가 아니었다. MVC 콘솔 컨트롤러는 원래 무겁다.

착각의 정체 - 스프링 웹 컨트롤러를 기준으로 생각했다

내가 했던 생각

“컨트롤러는 *View*와 도메인을 잇는 다리.
입력을 받아 넘기고 출력만 하면 되는,
그저 거쳐 가는 얇은 곳이다.”

콘솔 환경 MVC의 컨트롤러

스프링MVC의 컨트롤러에서
프론트엔드, 웹 서버로 대신 해주던 일까지 컨트롤러가 직접 한다.
입력 루프와 비즈니스 흐름 제어가 본래 그 몫(역할)이다.

같은 ‘컨트롤러’라는 이름, 다른 역할 범위

역할	스프링 웹 컨트롤러	콘솔 MVC 컨트롤러
입력 형식	HTTP 요청으로 정형화됨	Scanner로 직접 읽고 파싱
흐름 제어	요청-응답 단건, 루프 없음	while 루프로 게임 흐름 직접 제어
역할 크기	파싱 → 호출 → 응답, 얇음	프론트 + 웹서버 + 흐름 제어, 두꺼움

05

계층 개념 다시 세우기

MVC패턴, Repository와 Service는 계층은 무엇을 위해 왜 나왔는가

MVC 패턴 - 무엇을 위해 나왔는가

1970년대, UI와 비즈니스 로직이 뒤섞여 유지보수가 힘들다는 문제에서 출발했다.

MVC의 본질 UI와 도메인을 분리해 ‘변경’이 서로에게 번지지 않게 막는 것

Model	도메인 데이터 + 비즈니스 로직	시스템이 ‘무엇을 아는가’
View	화면 표현, 출력 중심	입력을 받는 곳이 아니다 -보여주는 역할
Controller	입력 해석 → Model 호출 → View 전달	분기·시나리오 결정 등 흐름 담당

오해 지점 - “입력 = View” 가 아니다. 입력을 받는 것은 Controller이며, View는 보여주는 역할이 핵심이다.

등장 순서 - **Repository**가 **Service**보다 먼저였다

계층은 한 번에 설계된 것이 아니라, '필요에 대한 압력신호'가 생길 때마다 하나씩 분리되어 나왔다.



1차 압력 — 영속성의 침범

도메인 규칙과 'DB에 어떻게 저장하나'가 한 클래스에 엉키면,
DB를 바꿀 때 도메인 코드가 흔들리고 테스트에 DB가 필요해진다.
→ Repository 분리.

내가 했던 착각

서비스 계층이 먼저 생긴 개념인 줄 알았다.
실제로는 영속성을 격리하는 Repository(2003)가 먼저 자리잡았고,
Service는 그 다음 압력의 산물이다.

Repository - 도메인 세계와 DB 세계의 번역기

존재 이유

도메인을 DB 기술로부터 격리한다.

도메인은 “JanggiGame을 id로 찾고 싶다”고만 말하고,

Repository가 그것을 SQL로 바꾸고 결과 행을 다시 도메인 객체로 재조립한다.

컬렉션처럼 보이는 인터페이스

쓰는 쪽은 마치 메모리 속 컬렉션을 다루는 것처럼 느낀다.

Repository는 “DB가 존재하지 않는 것처럼” 느끼게 해주는 환상을

주기 위한 추상화된 계층이다.

입출력 원칙

- 입력과 출력은 도메인 객체/값 객체/식별자만 사용
- DB 기술(ResultSet, Connection)은 밖으로 새지 않음
- 환경 객체(HttpServletRequest, Scanner)를 받지 않음

ReservationRepository (인터페이스)

```
public interface ReservationRepository {  
  
    List<Reservation> findAll();  
    Reservation save(Reservation reservation);  
    Optional<Reservation> findById(Long id);  
    void deleteById(Long id);  
  
    // 도메인 언어로만 말한다.  
    // SQL도, JdbcTemplate도 시그니처에 없다.  
}
```

JdbcReservationRepository - 구현체가
SQL, RowMapper, KeyHolder를 모두 흡수

06

서비스 계층의 필요성

“웹/콘솔 전환이 쉬워지려고?” - 그건 결과일 뿐 이유가 아니다

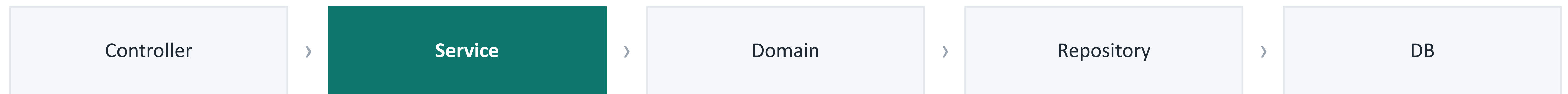
2차 압력 - 컨트롤러가 유스케이스 조율을 떠안는다

유스케이스 하나를 처리하려면 여러 도메인 객체와 Repository를 조율해야 하는데, 그것이 전부 컨트롤러로 몰린다.

- [1] 컨트롤러는 환경(콘솔·웹)에 묶이지만 유스케이스는 환경과 무관 → 환경 바꾸면 재사용 불가
- [2] 같은 유스케이스가 웹 API, 배치, CLI에서 필요할 때 코드 중복
- [3] 트랜잭션 경계를 컨트롤러에서 잡으면 HTTP 요청 단위에 묶어버림
- [4] 컨트롤러 테스트에 HTTP 목킹 등 환경 의존 준비가 필요

유스케이스 조율을 담당하는, 환경에 무관한 계층.
입력은 파라미터로 받고 출력은 반환값으로 돌려준다.

계층 흐름



‘웹 포팅이 쉬워진다’는 결과일 뿐이다. 근본 이유는 ?

진짜 이유 “변화의 이유가 다른 것들이 섞이면 고통스럽다(유지보수가 안좋고 변경에 약하다)” - 유스케이스를 환경으로부터 독립시키는 것

유스케이스는 ‘환경’이 아니라 ‘비즈니스’가 정의한다. 콘솔이든 웹이든 모바일이든 “새 게임을 시작한다”는 똑같다.

a

테스트 용이성

HTTP 요청이나 Scanner 없이 메서드만 호출하면 된다. 빠르고 간단하다.

b

유스케이스 재사용

같은 로직을 웹 API, 배치, CLI 터미널, 관리자 도구가 같은 Service로 호출한다.

c

트랜잭션 경계 명확화

‘유스케이스 하나 = 트랜잭션 하나’라는 자연스러운 매핑이 가능해진다.

d

의도가 코드로 드러남

Service 인터페이스만 봐도 ‘이 시스템이 할 수 있는 일’이 보인다.

07

해소된 네 가지 문제

컨트롤러에 뭉쳐 있던 유스케이스를 Service로 뽑아내자 한 번에 풀렸다

문제 1, 2 - 환경 종속과 유스케이스 중복

문제 1. 환경을 바꾸면 재사용 못 한다

BEFORE - 컨트롤러가 유스케이스 보유

```
private void startNewGame() {
    janggiGame = gameRepo.save(new JanggiGame());
    pieceRepo.saveAll(game.findGameId(), board);
    outputView.printGameStart(...); // 출력 섞임
} // 웹 포팅 시 통째로 다시 작성
```

AFTER - Service가 결과를 값으로 반환

```
public GameSnapshot startNewGame() {
    JanggiGame game = gameRepo.save(...);
    pieceRepo.saveAll(game.findGameId(), board);
    return new GameSnapshot(game, board);
} // 콘솔·웹 컨트롤러가 표현만 담당
```

문제 2. 유스케이스 중복

BEFORE - 진입점마다 같은 로직 복붙

```
JanggiController { /* 3줄 로직 */ }
AdminController { /* 위 3줄 복붙 */ }
BatchJob { /* 또 복붙 */ }
// 하나 바뀌면 전부 수정, 빠뜨리면 버그
```

AFTER - 한 곳의 Service를 호출

```
JanggiController { service.startNewGame(); }
AdminController { service.startNewGame(); }
DailyGameBatch { service.startNewGame(); }
// 로직은 한 곳 - DRY 원칙이 지켜진다
```

문제 3, 4 - 트랜잭션 경계와 테스트 환경 의존

문제 3. 트랜잭션 경계가 HTTP에 묶인다

“수를 둔다”는 기물 이동, 턴 변경, 종료 저장이 전부 성공하거나 전부 실패해야 한다.

Before 컨트롤러가 트랜잭션을 잡으면 경계가 ‘HTTP 요청 하나’에 묶인다.

After Service가 잡으면 경계가 ‘유스케이스 하나’ - 누가 호출하든 동일.

하나의 원리

분리 → 집중 → 독립

네 문제는 모두 ‘컨트롤러에 뭉친 유스케이스를 환경 무관 Service로 뽑아내는 것’ 하나로 풀린다.

전에는 유스케이스가 Controller안에 뒤섞여 암묵적으로 존재했는데, Service계층 도입으로 유스케이스가 1급 시민으로 코드에 명시적으로 등장.

문제 4. 테스트가 환경에 의존한다

BEFORE — 환경을 흉내내야 한다

```
System.setIn(new ByteArrayInputStream(...));
System.setOut(new PrintStream(output));
// 또는 MockMvc 설정 - 느리고 깨지기 쉬움
```

AFTER — 그냥 호출하면 된다

```
GameSnapshot snap = service.startNewGame();
assertThat(snap.gameId()).isNotNull();
assertThat(snap.board().findAll())
    .hasSize(32); // 빠르고 명확
```

08

서비스가 두꺼워지는 함정

Service로 뿔었다고 끝이 아니다 - 이번엔 Service가 비대해진다
도메인은 왜 아무것에도 의존하지 않는게 좋은가?

왜 두꺼워지나 -도메인을 데이터 통으로 쓰기 때문

1	도메인을 데이터 통처럼 사용	게터와 조건문이 많아지며 비즈니스 규칙이 도메인 바깥(Service)으로 샌다.
2	‘비즈니스 로직은 Service에’ 라는 오해	유스케이스 흐름 조율과 비즈니스 규칙을 구분하지 못하고 전부 Service에 둔다.
3	메시지 대신 직접 조작	객체에게 메시지를 보내지 않고 외부에서 내부 상태를 조립 -> 도메인이 약해진다.

트랜잭션 스크립트

Service가 일하고 객체는 데이터만 든다. 규칙이 흩어진다.

도메인 모델

객체가 일하고 Service는 흐름만 조립한다. 규칙이 응집된다.

도메인은 왜 아무것에도 의존하지 않아야 하는가

도메인은 '왜 이 소프트웨어가 존재하는가'의 답이고, 나머지는 그것을 구현하는 수단이다. 수단이 목적에 의존해야지, 목적이 수단에 의존하면 안 된다.

1

가장 안정적이어야 한다

비즈니스 규칙은 잘 안 바뀌지만
프레임워크, DB, UI는 자주 바뀐다.
자주 바뀌는 것이 안 바뀌는 것에 의존해야 한다.

2

순수해야 교체/테스트가 된다

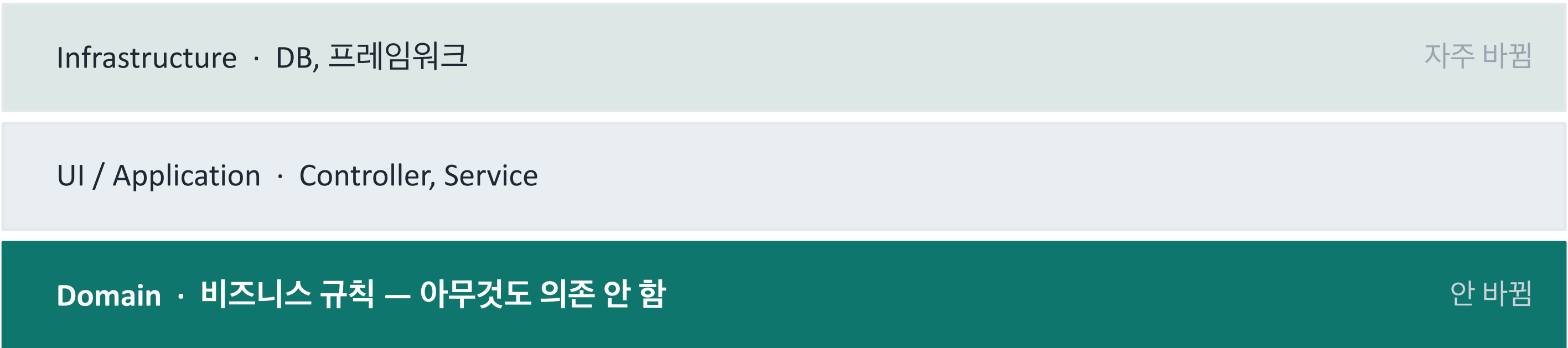
DB를 바꿔도, 웹→모바일→배치로 환경이 바뀌어도
도메인은 그대로.
프레임워크 없이 순수 유닛 테스트가 가능하다.

3

의존 방향 = 변경 전파 방향

A→B로 의존하면 B가 바뀔 때 A도 바뀐다.
도메인이 밖으로 의존하면 바깥의 변화가 안으로 스며든다.

의존은 항상 바깥에서 안으로만 흐른다



↓ 의존 방향

Clean, Hexagonal, DDD가 공통으로 말하는 의존성 역전의 최종 형태
-> 중심은 고요하고 바깥만 요동친다.

09

다시 바라보는 문제정의

무엇이 진짜 문제였는가 ?

“근거없는 감각이 아니라, 문제의 정체를 언어로 정의하기”

“근거없는 감각이 아니라, 문제의 정체를 언어로 정의하기”

처음의 진단

“컨트롤러가 그냥 무겁다. 코드를 줄여서 가볍게 만들면 된다.”

정확한 진단

“Service 계층이 없는 콘솔 MVC에서,
컨트롤러가 유스케이스 조율까지 떠안아 무거웠던 것이다.”

사고의 수정 ‘코드를 줄인다’가 아니라 ‘변화의 이유가 다른 책임을 제 계층으로 돌려보낸다’ - woowacourse/be-optional-rebuild레포 softmoca 브랜치의 방향.

한 계층에 서로 다른 ‘변화의 이유’가 섞이면, 그 계층은 쪼개진다

Repository 분리

‘DB 기술이 바뀌는 이유’ vs ‘도메인 규칙이 바뀌는 이유’

Service 분리

‘환경(HTTP,콘솔)이 바뀌는 이유’ vs ‘유스케이스가 바뀌는 이유’

10

서비스 입력 **Command DTO**의 발견

레벨 2 미션 - 웹환경에 콘솔 환경을 더하며 느낀 필요성을 깨닫다.

콘솔이 웹 **DTO**를 알아야 했다 -분리가 덜 된 신호

레벨 2 미션 1의 콘솔 환경 기능을 구현하며, Service가 Controller의 DTO를 그대로 받으면 콘솔이 웹 DTO를 알아야 하는 문제를 발견했다.

문제 - Service가 웹 DTO에 묶인다

```
public class UserService {
    public User createUser(
        SignupRequest request) {
        // @NotNull, @Email ... JSON 구조에
        // Service가 웹 환경에 묶여버린다.
    }
}
```

→ 배치·CLI가 웹 DTO를 억지로 생성해야 함

해결 - Service 전용 Command 정의

```
public record CreateUserCommand(
    String email, String name) { }

// Controller가 경계에서 변환
CreateUserCommand cmd =
    new CreateUserCommand(
        req.email().trim(), req.name());
service.createUser(cmd);
```

Command DTO가 실제로 해주는 것

- 이 유스케이스가 어떤 입력을 필요로 하는지 명시적으로 선언
- 여러 Controller(웹, 콘솔, 터미널, 배치)가 공유하는 입력 형식의 통일점
- Service 테스트에서 입력을 한 줄로 준비 가능

방탈출 미션에 적용 — 계층이 제자리를 지키는지 점검

각 계층의 ‘종속 표현’이 자기 안에만 있는지 점검 항목으로 만들었다.

 Controller	@RestController, @RequestBody, ResponseEntity 가 service-repository로 새지 않는다
 Repository	JdbcTemplate, RowMapper · SQL 문자열이 service,controller로 새지 않는다
 Service	‘여러 Repository 호출 + 정책 결정 + 조립’ 흐름이 컨트롤러로 흘러지지 않는다
 의존 방향	Controller → Service → Repository, 모두 Domain을 본다. 역방향 의존이 없다

분리가 잘 됐다면 -콘솔 UI를 추가할 때 Repository를 인터페이스로 바꾸고 메모리 구현체를 더하는 것만으로 끝나야 한다.
Service,Controller를 손대야 한다면 분리가 덜 된 것.

11

왜 ‘계층’ 개념이 혼란스러울까?

비슷한 개념에 붙여진 여러 다른 이름들.

계층 이름의 혼란 정리 - 핵심은 네 덩어리

여러 책과 커뮤니티가 비슷한 개념에 다른 이름을 붙였을 뿐, 핵심 네 덩어리는 공통이다.

계층	역할	프로젝트 예시	다른 이름들
UI / Presentation	외부 세계와 소통, 환경 의존	Controller · View	Presentation · Interface · Adapter
Application / Service	유스케이스 조율, 환경 무관, 트랜잭션 경계	BlackJackService · ReservationService	Service · Use Case
Domain	비즈니스 규칙과 핵심 개념, 무의존	Reservation · BlackJackGame · Theme	Business · Core · Entity
Infrastructure	DB·외부 API 등 기술 세부사항	JdbcRepository · DBConnectionManager	Persistence · Data Access

“Service 계층” ≈ “Application 계층” - 도메인은 ‘무엇이 존재하는가’, 애플리케이션은 ‘무엇을 할 수 있는가’를 다룬다.

정리

위화감 하나가 계층 전체를 다시 보게 했다

감각

“컨트롤러가 무겁다” - 전략,콜백으로 일단 비웠지만 더 어지러워졌다

인식

콘솔 MVC의 컨트롤러는 원래 무겁다. 웹 컨트롤러를 기준으로 착각했을 뿐

이해

Repository(1차), Service(2차)는 ‘변화의 이유’가 다를 때 분리되어 나왔다

수정

문제는 ‘무겁다’가 아니라 ‘유스케이스가 제 계층에 없다’였다

한 줄 요약 한 계층에 서로 다른 ‘변화의 이유’가 섞이면 그 계층은 쪼개진다.

감사합니다